

OHMC 2026 Workshop - Day 2

August 6th, 2026

Author: Veronica Roman-Reyna

Tutorial to Run QIIME 2 Interactively on the supercomputer

Topic: 16S rRNA amplicon sequencing analysis with QIIME 2 **Data type:** Paired-end Illumina reads (2×250, V4 region) **Source data:** Subset of the Mothur MiSeq SOP tutorial dataset **Reference workflow:** Adapted from the QIIME 2 Moving Pictures tutorial (<https://amplicon-docs.qiime2.org/en/stable/tutorials/moving-pictures/>)

What You'll Need

Before starting, make sure you have:

- This document open for instructions
- A second browser tab open to QIIME 2 View (<https://view.qiime2.org/>) — this is where you'll upload .qzv visualization files to inspect your results

What are .qza and .qzv files? QIIME 2 stores data and results in special zipped archive formats: .qza (“QIIME Zipped Artifact”) holds data/results for further processing, while .qzv (“QIIME Zipped Visualization”) holds interactive visualizations you view in a web browser. You'll upload .qzv files to view.qiime2.org (<https://view.qiime2.org/>) to see plots, tables, and interactive summaries.

Starting an Interactive Session

Request compute resources from the cluster's job scheduler (SLURM) so you have a dedicated space to run QIIME 2:

```
salloc -N 1 -n 12 -t 3:00:00 --account=open
```

What this does: | Flag | Meaning | |---| | -N 1 | Request 1 compute node | | -n 12 | Request 12 CPU tasks/cores | | -t 3:00:00 | Reserve the session for 3 hours | | --account=open | Bill the job to the open account |

⚠ Important: Once you're in an interactive session, **do not close the terminal window**. Closing it ends the session and erases everything held in memory — including any variables and activated environments you've set up. If it closes, you'll need to start over from this step.

Background

- **Raw data:** This is a subset of paired-end Illumina data (fastq files) from the Mothur MiSeq SOP tutorial, using 2×250 base pair sequencing of the 16S rRNA V4 region.
- **Commands:** The steps below are a modified version of the official QIIME 2 tutorial commands.

Note: This script processes **paired-end** reads (`sample_R1.fastq` and `sample_R2.fastq`), where each sample has two read files (forward and reverse). The same general workflow also works for **single-end** reads (`sample.fastq`), with minor changes to the import step.

Step 0: Define Variables

Setting a variable makes long file paths easier to reuse throughout the script instead of retyping them every time.

```
DAY2=/storage/group/one/default/workshop/2026/day2_16S/qiime2
```

- `$DAY2` points to the shared folder containing the workshop's input files (metadata, manifest, reference classifiers).
- `$WORK` is your **personal work directory** on the cluster, already defined in the supercomputer. This is where all the files you generate will be saved.

Remember: User-defined variables only exist for the current session. Every time you start a new interactive session, you'll need to define `$DAY2` again.

Step 1: Load QIIME 2

Load the software environment that contains QIIME 2 and its dependencies:

```
module load anaconda/2023.09
conda activate /storage/group/one/default/workshop/2026/envs/rachis-qiime2-2026.7
```

- `module load anaconda` makes the Anaconda package manager available.
- `conda activate` switches into the specific pre-built QIIME 2 environment for this workshop.

Note: This may take a minute to load. You'll know it worked when you see (rachis-qiime2-2026.7) appear at the beginning of your command line prompt — that confirms you're now working inside the QIIME 2 environment.

Step 2: Import Reads and Metadata

QIIME 2 needs your raw sequencing data and sample information converted into its own artifact format before it can do anything with them.

```
qiime tools import \
  --type 'SampleData[PairedEndSequencesWithQuality]' \
  --input-path $DAY2/data-manifest.txt \
  --output-path $WORK/pe-demux.qza \
  --input-format PairedEndFastqManifestPhred33V2

qiime metadata tabulate \
  --m-input-file $DAY2/metadata.tsv \
  --o-visualization $WORK/metadata-viz.qzv
```

Two supporting files are used here: - **data-manifest.txt** — tells QIIME 2 exactly where each sample's FASTQ files are located on the cluster. - **metadata.tsv** — contains information about your experiment (e.g., sample names, treatment groups, collection site).

The first command imports the demultiplexed paired-end reads into a `.qza` artifact (`pe-demux.qza`). The second command creates a visualization of your metadata table so you can double-check it looks correct.

Step 3: Summarize the Imported Data

```
qiime demux summarize \
  --i-data $WORK/pe-demux.qza \
  --o-visualization $WORK/pe-demux.qzv
```

This generates `pe-demux.qzv`, an interactive summary showing read counts per sample and sequence quality plots. **Upload this file to view.qiime2.org (<https://view.qiime2.org/>)** — the quality plots here are what you'll use in Step 4 to decide where to truncate your reads.

Step 4: Denoise Reads with DADA2

DADA2 filters out noisy/low-quality reads, corrects sequencing errors, removes chimeras, and merges paired-end reads into **ASVs (Amplicon Sequence Variants)** — high-resolution stand-ins for what used to be called OTUs.

```
qiime dada2 denoise-paired \
  --i-demultiplexed-seqs $WORK/pe-demux.qza \
  --p-trim-left-f 0 \
  --p-trunc-len-f 240 \
  --p-trim-left-r 0 \
  --p-trunc-len-r 160 \
  --o-representative-sequences $WORK/asv-seqs.qza \
  --o-table $WORK/asv-table.qza \
  --o-denoising-stats $WORK/denoising-stats.qza \
  --o-base-transition-stats $WORK/base-transition-stats.qza
```

Key parameters: - `--p-trim-left-f` / `--p-trim-left-r` : bases to trim from the *start* of forward/reverse reads (e.g., primers or low-quality leading bases). - `--p-trunc-len-f` / `--p-trunc-len-r` : length to *truncate* forward/reverse reads to, cutting off low-quality trailing bases (chosen based on the quality plots from Step 3).

Outputs: - `asv-table.qza` — a feature table of how many reads of each ASV appear in each sample. - `asv-seqs.qza` — the representative DNA sequence for each ASV. - `denoising-stats.qza` — a log of how many reads survived filtering, merging, and chimera removal at each stage (this is the one we'll focus on today). - `base-transition-stats.qza` — diagnostic output showing the error model DADA2 used to predict nucleotide substitutions (not covered in this workshop, but useful for troubleshooting).



This step is computationally heavy and may take **~6 minutes**. The terminal will appear to hang — that's normal, it's still working.

Tabulate the denoising stats so you can view them:

```
qiime metadata tabulate \
  --m-input-file $WORK/denoising-stats.qza \
  --o-visualization $WORK/denoising-stats.qzv
```

Upload `denoising-stats.qzv` to QIIME view to check how many reads were retained at each processing step.

~1–2 minutes.

Step 5: Visualize the Feature Table and Sequences

```
qiime feature-table summarize \
  --i-table $WORK/asv-table.qza \
  --o-feature-frequencies $WORK/feature-frequencies.qza \
  --o-sample-frequencies $WORK/sample-frequencies.qza \
  --o-summary $WORK/summary.qzv \
  --m-metadata-file $DAY2/metadata.tsv
```

Creates `summary.qzv`, an interactive view of how many reads/features each sample has — useful for spotting samples that may need to be excluded due to low depth. ~1–2 minutes.

```
qiime feature-table tabulate-seqs \
  --i-data $WORK/asv-seqs.qza \
  --o-visualization $WORK/asv-seqs.qzv
```

Creates `asv-seqs.qzv`, a browsable, BLAST-linkable list of the actual ASV sequences. 🕒 ~1–2 minutes.

Step 6: Build a Phylogenetic Tree

A phylogenetic tree gives structure to the data by showing the evolutionary relationships between the ASVs — this is required for phylogeny-based diversity metrics (like UniFrac) in the next step.

```
qiime phylogeny align-to-tree-mafft-fasttree \
  --i-sequences $WORK/asv-seqs.qza \
  --o-alignment $WORK/aligned-asv-seqs.qza \
  --o-masked-alignment $WORK/masked-aligned-asv-seqs.qza \
  --o-tree $WORK/unrooted-tree.qza \
  --o-rooted-tree $WORK/rooted-tree.qza
```

This single command performs four actions in sequence: multiple sequence alignment (MAFFT), masking of highly variable alignment positions, building an unrooted tree, and rooting that tree (FastTree). The final `rooted-tree.qza` is what you'll use downstream.

Step 7: Alpha and Beta Diversity

Alpha rarefaction — checks whether you've sequenced deeply enough to capture the full diversity within each sample:

```
qiime diversity alpha-rarefaction \
  --i-table $WORK/asv-table.qza \
  --i-phylogeny $WORK/rooted-tree.qza \
  --p-min-depth 10 \
  --p-max-depth 2500 \
  --m-metadata-file $DAY2/metadata.tsv \
  --o-visualization $WORK/alpha-rarefaction.qzv
```

`--p-min-depth` and `--p-max-depth` set the range of sequencing depths tested; the resulting curves should "level off" if sequencing depth was sufficient.

Core diversity metrics — calculates a standard suite of alpha diversity (within-sample) and beta diversity (between-sample) metrics all at once:

```
qiime diversity core-metrics-phylogenetic \
  --i-phylogeny $WORK/rooted-tree.qza \
  --i-table $WORK/asv-table.qza \
  --p-sampling-depth 2500 \
  --m-metadata-file $DAY2/metadata.tsv \
  --output-dir $WORK/core-metrics-results
```

--p-sampling-depth 2500 rarefies (subsamples) every sample down to 2,500 reads so diversity comparisons across samples are fair. Results are saved into a new folder, core-metrics-results , rather than individual named files.

Step 8: Taxonomic Classification

This step assigns a taxonomic identity (e.g., genus, species) to each ASV by comparing it against a reference database. We'll compare two commonly used reference databases: **SILVA** and **GTDB**.

Note: Always double check classifier versions — mismatched versions can cause errors or unreliable results.

SILVA classifier for 16S rRNA V4 region

```
qiime feature-classifier classify-sklearn \
  --i-classifier $WORK/silva-138-2-v4-classifier.qza \
  --i-reads $WORK/asv-seqs.qza \
  --o-classification $WORK/silva-taxonomy.qza

qiime metadata tabulate \
  --m-input-file $WORK/silva-taxonomy.qza \
  --o-visualization $WORK/silva-taxonomy.qzv

qiime taxa barplot \
  --i-table $WORK/asv-table.qza \
  --i-taxonomy $WORK/silva-taxonomy.qza \
  --m-metadata-file $DAY2/metadata.tsv \
  --o-visualization $WORK/silva-taxonomy-plots.qzv
```

GTDB classifier for V4 region

```
qiime feature-classifier classify-sklearn \
  --i-classifier $DAY2/gtdb-r232.0-2026.7.0-classifier.qza \
  --i-reads $WORK/asv-seqs.qza \
  --o-classification $WORK/gtdb-taxonomy.qza

qiime metadata tabulate \
  --m-input-file $WORK/gtdb-taxonomy.qza \
  --o-visualization $WORK/gtdb-taxonomy.qzv

qiime taxa barplot \
  --i-table $WORK/asv-table.qza \
  --i-taxonomy $WORK/gtdb-taxonomy.qza \
  --m-metadata-file $DAY2/metadata.tsv \
  --o-visualization $WORK/gtdb-taxonomy-plots.qzv
```

Each classifier block follows the same three-step pattern: 1. `classify-sklearn` — assigns taxonomy to each ASV using a pre-trained naive Bayes classifier. 2. `metadata tabulate` — creates a browsable table of the taxonomy assignments. 3. `taxa barplot` — creates an interactive stacked bar chart showing the relative taxonomic composition of each sample, viewable at multiple taxonomic levels (phylum, class, genus, etc.).

Comparing the SILVA and GTDB results side by side is a good exercise in how reference database choice can affect taxonomic classification outcomes.

Quick Reference: File Types

Extension	Meaning
.qza	QIIME artifact — data/results for further processing
.qzv	QIIME visualization — open at view.qiime2.org (https://view.qiime2.org/)
.tsv	Tab-separated metadata table
.txt (manifest)	Maps sample names to raw FASTQ file locations